

Programación en Lenguaje Ensamblador del Microcontrolador ATmega328P

1st Candela Vargas

5.645.897-6

Mercedes

2nd Neyzer Marcenal

5.529.314-9

Fray Bentos

3rd Julia Meléndrez

5.397.666-4

Fray Bentos

candela.vargas@estudiantes.utec.edu.uy neyzer.marcenal@estudiantes.utec.edu.uy julia.melendrez@estudiantes.utec.edu.uy

Abstract—En este laboratorio se desarrollaron diferentes aplicaciones utilizando el microcontrolador ATmega328P y programación en lenguaje ensamblador, con el objetivo de aplicar de forma práctica conceptos relacionados con la arquitectura AVR y el control de entradas y salidas. Se implementaron cuatro sistemas correspondientes al control de una lavadora automática, un contador digital con display de 7 segmentos, un sistema de selección y ejecución de ocho secuencias de LEDs y una Unidad Aritmético-Lógica (ALU) de 4 bits. Para el desarrollo de las actividades se utilizaron registros de propósito general, puertos digitales, operaciones aritméticas y lógicas, instrucciones de comparación y salto, desplazamientos de bits y retardos implementados mediante software. Los programas desarrollados fueron comprobados mediante simulaciones y posteriormente mediante las implementaciones correspondientes, verificando el comportamiento de las entradas, salidas y operaciones definidas para cada problema. Los resultados obtenidos permitieron comprobar el correcto funcionamiento de los sistemas desarrollados y relacionar la programación en lenguaje ensamblador con el control directo de los recursos del microcontrolador.

Index Terms—ATmega328P, lenguaje ensamblador, arquitectura AVR, microcontroladores, puertos digitales, ALU.

I. INTRODUCCIÓN

Los microcontroladores son dispositivos ampliamente utilizados en sistemas electrónicos debido a su capacidad para procesar información y controlar diferentes periféricos mediante software. En este laboratorio se trabaja con el microcontrolador **ATmega328P**, con el propósito de comprender de forma práctica algunos de sus principales recursos y su programación mediante lenguaje ensamblador.

A lo largo de la actividad se desarrollan diferentes problemas que permiten trabajar con los **registros de propósito general, puertos de entrada y salida, registro de estado SREG, operaciones aritméticas y lógicas, instrucciones de salto y retardos implementados mediante software**, buscando relacionar la programación de bajo nivel con el comportamiento físico del microcontrolador.

Para esto, el laboratorio plantea la implementación de cuatro sistemas: el control de una **lavadora automática, un contador digital con display de 7 segmentos**, un sistema de **selección y ejecución de secuencias mediante LEDs** y una **Unidad Aritmético-Lógica (ALU)**. Cada problema permite aplicar diferentes conceptos de la arquitectura AVR y comprender la interacción entre el programa desarrollado y los dispositivos conectados al microcontrolador.

II. OBJETIVOS

A. Objetivo General

Desarrollar e implementar soluciones en lenguaje ensamblador para el microcontrolador ATmega328P, aplicando los principales recursos de la arquitectura AVR para el control de entradas, salidas y diferentes operaciones mediante sistemas digitales.

B. Objetivos Específicos

- **Configurar y utilizar los puertos de entrada y salida del ATmega328P**, permitiendo la interacción con pulsadores, LEDs y displays utilizados en las diferentes actividades del laboratorio.
- **Aplicar instrucciones aritméticas, lógicas, de comparación, salto y desplazamiento**, junto con el uso de registros y retardos por software, para implementar el comportamiento requerido en cada problema.
- **Comprobar el funcionamiento de los programas desarrollados mediante simulación e implementación física**, verificando la respuesta del microcontrolador ante las diferentes entradas y condiciones establecidas.

III. MATERIALES UTILIZADOS

- Arduino Uno con microcontrolador ATmega328P.
- Protoboard.
- LEDs.
- Resistencias.
- Pulsadores.
- DIP-switches
- Cables de conexión.
- Cable USB para programación y alimentación.

IV. SOFTWARE UTILIZADO

- Microchip Studio 7.
- PICSIMLAB.
- AVR Assembler.
- Git y GitHub.

V. MARCO TEÓRICO

A. Microcontrolador ATmega328P

El ATmega328P es un microcontrolador de 8 bits perteneciente de la familia AVR. Está basado en una arquitectura RISC, la cual utiliza un conjunto de instrucciones

optimizado para realizar las operaciones de forma rápida y eficiente. Una de sus características principales es que cuenta con 32 registros de propósito general de 8 bits, los cuales se encuentran directamente relacionados con la Unidad Aritmético-Lógica (ALU). Esto permite que gran parte de las instrucciones puedan ejecutarse en un único ciclo de reloj [1].

En cuanto a la memoria, el ATmega328P dispone de **32 kB de memoria Flash** para almacenar el programa, **2 kB de memoria SRAM** para el manejo temporal de datos durante la ejecución y **1 kB de memoria EEPROM**, utilizada para almacenar información que debe conservarse incluso cuando el microcontrolador deja de recibir alimentación [1].

El microcontrolador también dispone de diferentes recursos de entrada y salida. Entre ellos se encuentran **23 líneas de entrada/salida (I/O) programables**, temporizadores/contadores, interrupciones internas y externas, comunicación USART, SPI y una interfaz serial de dos hilos, además de un conversor analógico-digital (ADC). Estos recursos permiten que el microcontrolador pueda comunicarse e interactuar con diferentes dispositivos externos [1].

B. Arquitectura AVR, registros y SREG

La arquitectura AVR del ATmega328P utiliza **32 registros de propósito general de 8 bits**, identificados desde R0 hasta R31. Estos registros están conectados directamente con la Unidad Aritmético-Lógica (ALU), permitiendo almacenar temporalmente los datos utilizados durante la ejecución de las instrucciones y realizar operaciones aritméticas y lógicas de manera eficiente. Además, los seis últimos registros pueden agruparse de a pares para formar los registros de 16 bits **X**, **Y** y **Z**, utilizados principalmente para el direccionamiento de datos [1].

El microcontrolador también cuenta con el **registro de estado SREG (Status Register)**, formado por ocho bits que funcionan como banderas y reflejan diferentes condiciones producidas durante las operaciones realizadas por la ALU. Estas banderas permiten conocer características del resultado de una operación y son utilizadas posteriormente por diferentes instrucciones del programa para tomar decisiones [1].

Entre las banderas del SREG se encuentran **Carry (C)**, **Zero (Z)** y **Negative (N)**. La bandera **C** indica la presencia de un acarreo en una operación aritmética o lógica; la bandera **Z** indica que el resultado de una operación fue cero; y la bandera **N** refleja un resultado negativo. Estas banderas son especialmente útiles cuando se realizan operaciones aritméticas, comparaciones y saltos condicionales [1].

C. Puertos digitales GPIO

Los puertos **GPIO (General Purpose Input/Output)** permiten que el microcontrolador se comunique con elementos externos mediante señales digitales. Los pines pueden configurarse individualmente como entradas o salidas dependiendo de la función que deban cumplir dentro del circuito. En el ATmega328P, los puertos disponen además de resistencias pull-up internas que pueden activarse mediante software [1].

Para controlar el funcionamiento de los pines se utilizan principalmente tres registros: **DDRx**, **PORTx** y **PINx**, donde la letra x representa el puerto utilizado, por ejemplo B, C o D. El registro **DDRx (Data Direction Register)** establece la dirección de cada pin: un bit configurado en 1 funciona como salida, mientras que un bit en 0 funciona como entrada. **PORTx** permite establecer el nivel lógico de los pines configurados como salidas y también interviene en la activación de las resistencias pull-up cuando el pin funciona como entrada. Por otra parte, **PINx** permite leer el estado lógico presente en los pines de entrada.

D. Lenguaje ensamblador AVR

El **lenguaje ensamblador AVR** es un lenguaje de programación de bajo nivel que permite trabajar de forma directa con los recursos internos del microcontrolador, como los registros, los puertos de entrada y salida y los bits individuales. A diferencia de los lenguajes de alto nivel, las instrucciones en ensamblador están estrechamente relacionadas con las operaciones que realiza el procesador, lo que permite tener un mayor control sobre el funcionamiento del hardware.

El conjunto de instrucciones AVR permite realizar diferentes tipos de operaciones. Entre ellas se encuentran las instrucciones de **carga y transferencia de datos**, utilizadas para colocar valores en registros o enviarlos hacia los puertos; las instrucciones **aritméticas y lógicas**, que permiten realizar cálculos y operaciones entre bits; y las instrucciones de **comparación y salto**, utilizadas para modificar el flujo del programa dependiendo de determinadas condiciones. La ALU de AVR se encuentra conectada directamente con los 32 registros de propósito general y permite realizar operaciones aritméticas, lógicas y sobre bits [1].

También existen instrucciones que permiten actuar directamente sobre bits individuales de los registros de entrada y salida. Por ejemplo, **SBI** y **CBI** permiten establecer o limpiar un bit sin modificar los demás bits del mismo puerto, facilitando la configuración y el control individual de los pines [1].

Durante el desarrollo del laboratorio se utilizaron distintas instrucciones de ensamblador AVR, como **LDI** para cargar valores en los registros, **OUT** para escribir datos en los puertos, **CPI** para realizar comparaciones, **RJMP** y **BRNE** para controlar el flujo del programa y **SBI** y **CBI** para modificar bits individuales.

E. Retardos por software

Los **retardos por software** son una técnica utilizada para generar una espera determinada durante la ejecución de un programa. En lenguaje ensamblador pueden implementarse mediante ciclos o bucles que repiten una serie de instrucciones una determinada cantidad de veces antes de continuar con la ejecución normal del programa.

La duración del retardo depende principalmente de la **frecuencia de reloj del microcontrolador**, de la cantidad de ciclos del reloj que necesita cada instrucción para ejecutarse y del número de veces que se repiten los bucles. En la arquitectura AVR, las instrucciones presentan tiempos de ejecución

definidos en ciclos de reloj, por lo que es posible estimar el tiempo total de un retardo teniendo en cuentaa estos valores.

En este laboratorio se utilizaron retardos por software mediante **bucles anidados y registros utilizados como contadores**. Los registros se cargan incilamente con determinados valores y luego se decrementen hasta alcanzar una condición que permita finalizar el bucle. De esta manera se genera una pausa entre las diferentes acciones realizadas por el microcontrolador, permitiendo, por ejemplo, que los cambios producidos en las ecuencias de LEDs puedan ser apreciados visualmente

F. Máquinas de estados

Una **máquina de estados** es una forma de organizar el funcionamiento de un sistema mediante un conjunto de estados definidos. En cada momento, el sistema se encuentra en un determinado **estado**, el cual representa la situación o tarea que está realizando. El cambio de un estado a otro se denomina **transición** y ocurre cuando se cumple una determinada **condición de transición**, como puede ser la activación de una entrada, la pulsación de un botón o la finalización de un proceso [2].

Este método permite dividir un funcionamiento complejo en etapas más simples y establecer de manera ordenada qué debe realizar el sistema en cada una de ellas. De esta forma, el comportamiento del programa depende tanto del estado en el que se encuentra actualmente como de las condiciones necesarias para avanzar hacia el siguiente [2].

En este laboratorio, el concepto de máquina de estados se aplica principalmente al **control de la lavadora**, donde cada etapa del proceso puede representarse mediante un estado diferente. El programa debe avanzar entre estos estados según las condiciones establecidas, permitiendo controlar de forma secuencial las distintas etapas del funcionamiento de la lavadora.

G. Unidad Aritmético-Lógica (ALU)

La Unidad Aritmético-Lógica (ALU) es la parte del microcontrolador encargada de realizar las operaciones aritméticas y lógicas necesarias durante la ejecución de un programa. En la arquitectura AVR, la ALU se encuentra conectada directamente con los **32 registros de propósito general**, lo que permite realizar distintas operaciones entre registros o entre un registro y un valor inmediato de manera eficiente [1].

Las operaciones realizadas por la ALU pueden dividirse principalmente en operaciones aritméticas, lógicas y operaciones sobre bits. Entre ellas se encuentran la suma, resta e incremento, además de operaciones lógicas como **AND**, **OR** y **XOR** y operaciones de desplazamiento de bits. Estas permiten realizar cálculos y modificar datos según las necesidades del programa [1].

Los resultados obtenidos mediante estas operaciones pueden modificar las diferentes banderas del registro **SREG**, permitiendo conocer determinadas características del resultado y

utilizar esta información para controlar el flujo del programa mediante operaciones condicionales [1].

En las actividades desarrolladas durante el laboratorio, las operaciones de la ALU se utilizan para realizar cálculos, comparaciones y manipulación de bits. Además, instrucciones como los desplazamientos permiten modificar patrones binarios, mientras que las operaciones aritméticas permiten incrementar o decrementar valores utilizados durante la ejecución de los programas.

H. Display de 7 segmentos

El **display de 7 segmentos** es un dispositivo utilizado para representar números mediante siete segmentos luminosos, identificados generalmente de la **a** a la **g**. Cada número se obtiene encendiendo una combinación determinada de estos segmentos [4].

Para facilitar su control mediante un microcontrolador, cada número puede asociarse a un **patrón binario** que determina qué segmentos deben encenderse o apagarse. Estos patrones pueden almacenarse en una **tabla de consulta o LUT (Look-Up Table)**, permitiendo obtener directamente la combinación correspondiente al número que se desea mostrar.

I. Antirrebote de pulsadores

Los **pulsadores mecánicos** pueden generar pequeños cambios rápidos entre los niveles lógicos 0 y 1 en el momento de ser presionados o liberados. Este fenómeno se conoce como rebote y puede provocar que el microcontrolador interprete una única pulsación como varias [3].

Para evitar estas detecciones incorrectas se utiliza el **antirrebote (debounce)**, que puede implementarse mediante hardware o software. En este laboratorio se realiza mediante software, evitando que una misma pulsación genere múltiples acciones dentro del programa.

J. Máscaras y desplazamientos de bits

Las **máscaras de bits** permiten seleccionar, modificar o comprobar bits específicos dentro de un dato binario sin afectar necesariamente al resto. Para ello se utilizan principalmente operaciones lógicas como **AND**, **OR** y **XOR**, dependiendo de la acción que se quiera realizar.

Por otro lado, los **desplazamientos de bits** permiten mover los bits de un registro hacia la izquierda o hacia la derecha. En AVR pueden utilizarse instrucciones como **LSL (Logical Shift Left)** y **LSR (Logical Shift Right)** para realizar estos desplazamientos.

En este laboratorio, estas operaciones se utilizaron principalmente para la manipulación de datos y la generación de patrones. Por ejemplo, en las secuencias de LEDs, los desplazamientos permiten mover progresivamente un bit entre las distintas posiciones, generando visualmente el desplazamiento del LED encendido.

VI. METODOLOGÍA

A. Lavadora Automática

1) *Diseño general del sistema:* Para el desarrollo de la lavadora automática se organizó el funcionamiento del programa mediante una máquina de estados. De esta forma, el proceso se dividió en diferentes etapas y se establecieron las condiciones necesarias para pasar de una a otra.

Al iniciar el sistema, la lavadora queda en estado de espera y se enciende el LED de listo. Antes de comenzar el ciclo, el usuario puede seleccionar entre carga ligera, media o pesada utilizando un único pulsador. Luego de presionar START, el programa verifica que la puerta se encuentre cerrada y posteriormente espera que el sensor de nivel indique que se alcanzó la cantidad de agua necesaria.

Una vez cumplidas estas condiciones, el sistema ejecuta automáticamente las etapas de lavado, centrifugado y secado. Los LEDs permiten identificar el estado actual de la lavadora, mientras que el movimiento del motor se representó mediante dos LEDs adicionales para indicar el giro hacia la derecha y hacia la izquierda.

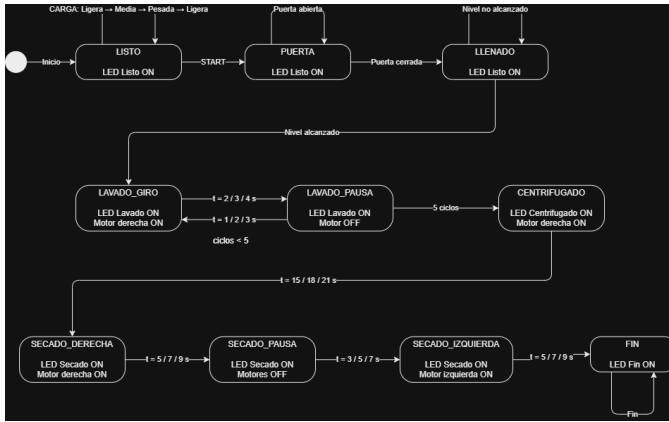


Fig. 1. Máquina de estados utilizada para el control de la lavadora automática.

La Fig. 1 representa el recorrido general del programa. Los estados iniciales permiten seleccionar la carga y verificar las condiciones de puerta y nivel de agua. Una vez iniciado el lavado, las transiciones siguientes dependen principalmente de la finalización de los tiempos correspondientes a cada etapa.

2) *Asignación de entradas y salidas:* Para implementar el sistema se utilizaron los puertos B, C y D del ATmega328P. Los puertos B y C se utilizaron como salidas para los LEDs de estado, selección de carga y representación del motor. Por otra parte, los pines PD2 a PD5 fueron configurados como entradas para los pulsadores y sensores.

En las entradas se habilitaron las resistencias pull-up internas del microcontrolador. Por este motivo, una entrada sin activar presenta un nivel lógico alto, mientras que al presionar un pulsador o activar un sensor el nivel pasa a bajo.

La asignación completa de pines se muestra en la Tabla I. Esta distribución permitió mantener separadas las entradas del sistema de los LEDs utilizados para representar los distintos estados.

TABLE I
ASIGNACIÓN DE ENTRADAS Y SALIDAS DE LA LAVADORA

ATmega328P	Arduino Uno	Función
PB0	D8	LED Listo
PB1	D9	LED Lavado
PB2	D10	LED Centrifugado
PB3	D11	LED Secado
PB4	D12	LED Fin
PB5	D13	LED Carga ligera
PC0	A0	LED Carga media
PC1	A1	LED Carga pesada
PC2	A2	Motor / giro derecha
PC3	A3	Motor / giro izquierda
PD2	D2	Pulsador START
PD3	D3	Pulsador selección de carga
PD4	D4	Sensor de puerta
PD5	D5	Sensor de nivel de agua

3) *Selección de carga y temporización:* La selección de carga se almacenó en un registro utilizando los valores 0, 1 y 2 para representar carga ligera, media y pesada, respectivamente. Cada pulsación del botón de carga incrementa este valor y actualiza el LED correspondiente. Al superar la carga pesada, la selección vuelve nuevamente a ligera.

El mismo valor se utiliza posteriormente para modificar los tiempos de funcionamiento. De esta manera, no fue necesario crear una rutina diferente para cada tipo de carga, sino que los tiempos se calculan a partir de la selección realizada.

TABLE II
TIEMPOS IMPLEMENTADOS SEGÚN EL TIPO DE CARGA

Carga	Giro L.	Pausa L.	Centrif.	Der.	Pausa S.	Izq.
Ligera	2 s	1 s	15 s	5 s	3 s	5 s
Media	3 s	2 s	18 s	7 s	5 s	7 s
Pesada	4 s	3 s	21 s	9 s	7 s	9 s

Como se observa en la Tabla II, durante el lavado se aumenta un segundo en el giro y en la pausa para cada nivel de carga. El procedimiento se repite cinco veces. En el centrifugado se parte de 15 segundos y se agregan 3 segundos según la carga. Para el secado se realiza un giro a la derecha, una pausa y un giro hacia la izquierda, aumentando 2 segundos en cada tiempo según la selección.

La consigna indica que la cantidad de ciclos de secado puede variar según la carga, pero no establece una cantidad concreta de repeticiones para cada caso. Por este motivo, se mantuvo una secuencia de derecha-pausa-izquierda y se realizó el ajuste mediante los tiempos.

4) *Implementación en lenguaje ensamblador:* El programa fue desarrollado completamente en lenguaje ensamblador para el ATmega328P. Se utilizaron registros de propósito general para almacenar valores temporales, la carga seleccionada, la cantidad de segundos de cada etapa y el número de repeticiones realizadas durante el lavado.

La estructura se organizó mediante etiquetas que representan los diferentes estados del sistema. Las instrucciones SBI y CBI se utilizaron para encender y apagar los LEDs, mientras que SBIS permitió comprobar el estado de los pulsadores

y sensores. También se emplearon instrucciones como INC, DEC, CPI, BREQ, BRNE y RJMP para realizar comparaciones, conteos y cambios en el flujo del programa.

```

LISTO:
    SBIS PIND, PD3      ; Si PD3 está en 1, no se presionó CARGA
    RJMP CAMBIAR_CARGA  ; Si está en 0, cambia la carga

    SBIS PIND, PD2      ; Si PD2 está en 1, no se presionó START
    RJMP ESPERA_START   ; Si está en 0, se presionó START

    RJMP LISTO          ; Si no se presionó nada, sigue esperando

ESPERA_START:
    SBIS PIND, PD2      ; Revisa si ya soltamos START
    RJMP ESPERA_START   ; Si sigue en 0, sigue esperando

    RJMP PUERTA         ; Cuando vuelve a 1, revisamos la puerta

PUERTA:
    SBIS PIND, PD4      ; PD4=1 significa puerta abierta
    RJMP LLENADO        ; PD4=0 significa puerta cerrada

    RJMP PUERTA         ; Si está abierta, se queda esperando

LLENADO:
    SBIS PIND, PD5      ; PD5=1 significa que todavía falta agua
    RJMP PREPARAR_LAVADO ; PD5=0 significa nivel alcanzado

    RJMP LLENADO        ; Mientras no esté lleno, sigue esperando

```

Fig. 2. Lectura de pulsadores y sensores durante los estados iniciales de la lavadora.

En la Fig. 2 se muestra la lectura de las entradas principales. El programa permanece esperando mientras no se cumpla la condición correspondiente y utiliza saltos para avanzar al siguiente estado cuando se detecta START, la puerta cerrada y finalmente el nivel de agua alcanzado.

Para controlar las cinco repeticiones del lavado se utilizó un registro como contador. En cada repetición se ejecuta primero el giro durante el tiempo correspondiente a la carga y luego una pausa. Al finalizar la pausa se decrementa el contador y, mientras su valor sea distinto de cero, el programa vuelve al estado de giro.

5) *Temporización mediante Timer1*: Para generar los tiempos de las diferentes etapas se utilizó el Timer1 del ATmega328P. Este temporizador es de 16 bits y fue configurado con un prescaler de 256. Para una frecuencia de reloj de 16 MHz se utilizó el valor inicial 3036 en el contador, obteniendo un desbordamiento aproximadamente cada segundo.

La bandera de desbordamiento TOV1 es comprobada por el programa mediante polling. A partir de esta base de un segundo se creó la subrutina RETARDO_SEGUNDOS, que repite la espera según el valor almacenado en el registro segundos.

Debido al uso de subrutinas mediante RCALL y RET, también se configuró el Stack Pointer al inicio del programa utilizando SPH, SPL y RAMEND.

```

RETARDO_SEGUNDOS:
    RCALL RETARDO_1S    ; Espera un segundo

    DEC segundos        ; Resta un segundo del total
    BRNE RETARDO_SEGUNDOS ; Si todavía quedan segundos, repite

    RET                 ; Si llegó a 0, vuelve al programa

RETARDO_1S:
    SBI TIFR1, TOV1     ; Limpia la bandera de desbordamiento anterior

    LDI temp, HIGH(3036) ; Parte alta del valor inicial del Timer1
    STS TCNT1H, temp

    LDI temp, LOW(3036)  ; Parte baja del valor inicial
    STS TCNT1L, temp

ESPERA_TIMER1:
    SBIS TIFR1, TOV1     ; ¿Timer1 ya llegó al desbordamiento?
    RJMP ESPERA_TIMER1   ; No: sigue esperando

    RET                 ; Si: pasó aproximadamente 1 segundo

```

Fig. 3. Subrutinas utilizadas para generar los retardos mediante Timer1.

La Fig. 3 muestra la rutina utilizada como base para las temporizaciones del sistema. El uso de Timer1 permitió utilizar los tiempos reales establecidos para lavado, centrifugado y secado.

6) *Simulación e implementación*: Una vez finalizado el programa, se ensambló en Microchip Studio y se verificó que no presentara errores. El archivo .hex generado se cargó posteriormente en PICSIMLAB utilizando una placa Arduino Uno virtual.

En la simulación se agregaron los LEDs, pulsadores y sensores necesarios para representar todas las entradas y salidas. Primero se comprobó el cambio entre carga ligera, media y pesada. Luego se verificó el ciclo completo desde START hasta el estado de fin, incluyendo la detección de puerta, nivel de agua, lavado, centrifugado y secado.

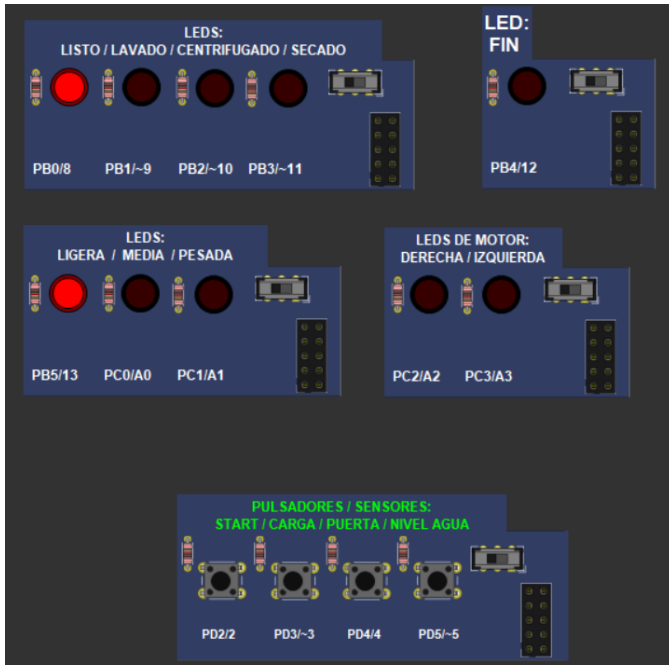


Fig. 4. Simulación completa de la lavadora automática en PICSimLab.

La Fig. 4 muestra la distribución utilizada durante la simulación. Los LEDs permiten visualizar tanto la carga seleccionada como el estado actual y el sentido de giro representado para el motor.

Luego de comprobar el programa mediante simulación se realizó la prueba del sistema implementado, verificando nuevamente el comportamiento de las entradas, los LEDs y la secuencia completa.

7) *Versionado del programa*: El desarrollo del código fue gestionado mediante Git y GitHub. Los cambios se fueron separando en diferentes etapas para mantener un registro de la evolución del programa y de las funciones incorporadas durante el desarrollo.

TABLE III
ETAPAS DE DESARROLLO REGISTRADAS MEDIANTE GIT

Commit	Modificación realizada
1	Configuración de GPIO, Stack y selector de carga
2	Configuración de Timer1 y rutinas de retardo
3	Incorporación del secado y estado final
4	Incorporación del centrifugado según la carga
5	Integración de la máquina de estados y ciclo completo

La Tabla III resume las principales etapas registradas en el repositorio. El código completo y el historial de modificaciones se encuentran disponibles mediante el enlace de GitHub incluido en el Apéndice.

B. Contador digital

1) *Desarrollo del Código en Lenguaje Ensamblador (ASM)*: Se procedió con el diseño del software programado exclusivamente en lenguaje ensamblador (ASM) para el microcontrolador ATmega328P (Arduino Uno):

- **Configuración de puertos I/O**: Se definieron los registros de dirección de datos (DDR_x) para establecer los pines de entrada asociados a los tres pulsadores (incremento, decremento y *reset*) y los pines de salida dedicados al control del display.
- **Tabla de conversión (Look-up Table)**: Se codificó una tabla de búsqueda asignando el valor hexadecimal correspondiente a cada dígito numérico (0 a 9). Al tratarse de un display de **cátodo común**, los valores en hexadecimal definen los '1' lógicos (altos) necesarios para activar directamente cada segmento.
- **Lógica del contador y anti-rebote**: Se desarrollaron las rutinas de lectura de los botones, incorporando algoritmos de tiempo (retardos por software en ASM) para la eliminación del efecto de rebote mecánico (*debouncing*), así como las subrutinas de incremento, decremento y reinicio del contador, asegurando el control de límites de conteo.

TABLE IV
VALORES HEXADECIMALES PARA EL DISPLAY DE CÁTODO COMÚN (0 A 9).

Número	Valor Hexadecimal
0	0x3F
1	0x06
2	0x5B
3	0x4F
4	0x66
5	0x6D
6	0x7D
7	0x07
8	0x7F
9	0x6F

La lógica de control implementada en firmware se ilustra en el diagrama de flujo de la Fig. 5. En este se detalla el proceso desde la inicialización del sistema (configuración de puertos, registros y puntero de pila) hasta la verificación cíclica de los botones de entrada (PB0, PB1 y PB2), asegurando el correcto incremento, decremento, manejo de límites (0–9) y el filtrado por software de los rebotes mecánicos.

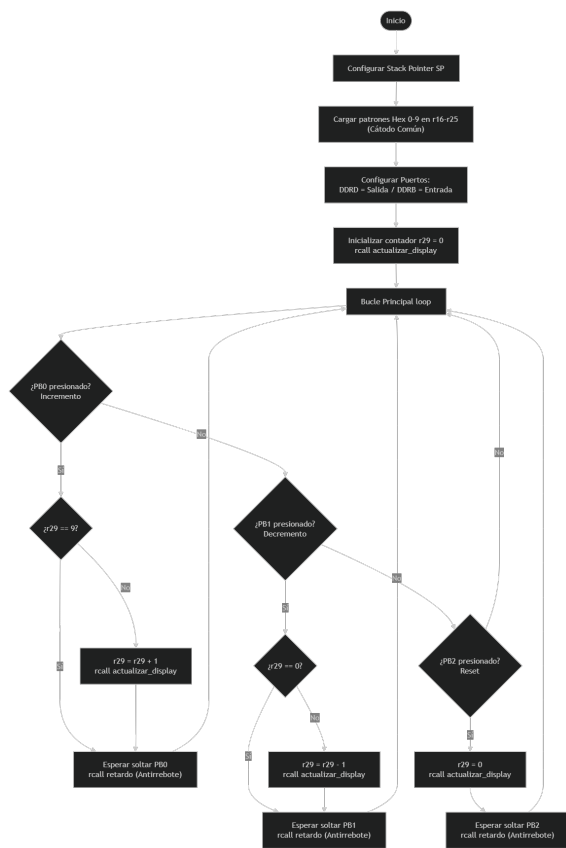


Fig. 5. Diagrama de flujo del contador en ensamblador.

2. *Simulación del Sistema en PicSimLab:* Previo al ensamblaje del hardware, el archivo ejecutable (.hex) obtenido tras la compilación del código ASM se cargó en el entorno de simulación PicSimLab:

- Se configuró la arquitectura virtual seleccionando la placa Arduino Uno, el display de 7 segmentos de cátodo común y los tres botones de entrada.
- Se validó el correcto funcionamiento de las rutinas de conteo ascendente, descendente y reinicio.
- Se comprobó la visualización correcta de los dígitos en hexadecimal en el display y la efectividad del filtrado de rebote ante eventos de entrada.

3. *Montaje e Implementación Física:* Una vez verificado el comportamiento del sistema en el entorno virtual, se procedió con la implementación en hardware real:

- Se realizó el conexionado de los tres pulsadores y el display de 7 segmentos de cátodo común al Arduino Uno, conectando su pin común a tierra (GND) e integrando resistencias limitadoras de corriente para los segmentos.
- Se programó la placa Arduino Uno con el firmware compilado.
- Se realizaron pruebas experimentales en tiempo real para verificar la respuesta precisa del contador ante las pulsaciones físicas y la estabilidad de la visualización en el display.

C. Selección y ejecución de secuencias

Para el desarrollo del sistema de secuencias se utilizaron ocho LEDs como salidas digitales y tres pulsadores como entradas. El programa fue estructurado mediante diferentes subrutinas, separando la lectura de los pulsadores, la selección de la secuencia, la generación de los patrones luminosos y los retardos. De esta manera, el programa ejecuta continuamente una de las ocho secuencias disponibles hasta detectar una nueva acción por parte del usuario.

```

7  INICIO:
8  ldi r16, 0xFF
9  out DDRD, r16
10
11  ; PB0, PB1 y PB2 como entradas
12  cbi DDRB, PB0
13  cbi DDRB, PB1
14  cbi DDRB, PB2
15
16  ; Pull-up interno de los tres botones
17  sbi PORTB, PB0
18  sbi PORTB, PB1
19  sbi PORTB, PB2
20
21  ; Comienza por la Secuencia 1
22  ldi r21, 1
23
24  rjmp SELECTOR

```

Fig. 6. Configuración inicial de los puertos y del selector de secuencia.

Como se observa en la Fig. 6, inicialmente se configuró el **PORTD** como salida mediante el registro **DDRD**, permitiendo utilizar sus ocho bits para el control de los LEDs. Por otra parte, los bits PB0, PB1 y PB2 del **PORTB** fueron configurados como entradas para los tres pulsadores y se habilitaron sus resistencias pull-up internas. El registro **r21** se utilizó para almacenar el número de la secuencia activa, inicializándose con el valor 1.

```

30     SELECTOR:
31         cpi r21, 1
32         brne SELECTOR2
33         rjmp SECUENCIA1
34
35     SELECTOR2:
36         cpi r21, 2
37         brne SELECTOR3
38         rjmp SECUENCIA2
39
40     SELECTOR3:
41         cpi r21, 3
42         brne SELECTOR4
43         rjmp SECUENCIA3
44
45     SELECTOR4:
46         cpi r21, 4
47         brne SELECTOR5
48         rjmp SECUENCIA4

```

Fig. 7. Selección de la secuencia mediante el registro r21.

```

74     SECUENCIA1:
75         ldi r17, 0b00000001
76
77     BUCLE_S1:
78         out PORTD, r17
79         rcall RETARDO
80
81         rcall LEER_BOTONES
82         cpi r21, 1
83         brne IR_SELECTOR_S1
84
85         lsl r17
86
87         ;si llegó a cero, reiniciar patrón
88         brne BUCLE_S1
89
90         rjmp SECUENCIA1
91
92     IR_SELECTOR_S1:
93         rjmp SELECTOR

```

Fig. 8. Implementación de la secuencia 1 mediante desplazamiento de bits.

Como se observa en la Fig. 7, la selección de las secuencias se realizó utilizando el registro **r21**, cuyo contenido indica cuál de las ocho secuencias debe ejecutarse. Mediante la instrucción **CPI** se compara su valor con el número correspondiente a cada secuencia. Si los valores no coinciden, **BRNE** continúa con la siguiente comparación; en caso contrario, **RJMP** dirige la ejecución hacia la rutina correspondiente.

Como se observa en la Fig. 8, en la secuencia 1 se inicializa el registro **r17** con el patrón binario 00000001. Este valor se envía al **PORTD**, encendiendo inicialmente un único LED. Después de cada retardo, la instrucción **LSL** desplaza el contenido de **r17** una posición hacia la izquierda, generando el movimiento progresivo del LED encendido. Cuando el patrón llega a cero, la rutina vuelve a inicializarse y comienza nuevamente desde el primer LED.


```

367     LEER_BOTONES:
368         sbic PINB, PB0
369         rjmp REVISAR_BOTON2
370
371         inc r21
372
373         ; Si pasó de 8, volver a 1
374         cpi r21, 9
375         brne ESPERAR_B1
376
377         ldi r21, 1
378
379     ESPERAR_B1:
380         sbis PINB, PB0
381         rjmp ESPERAR_B1
382
383         ret
384
385     REVISAR_BOTON2:
386         sbic PINB, PB1
387         rjmp REVISAR_BOTON3
388
389         dec r21
390
391         ; Si bajó de 1, volver a 8
392         cpi r21, 0
393         brne ESPERAR_B2
394
395         ldi r21, 8

```

Fig. 9. Rutina de lectura de los pulsadores y actualización de la secuencia.

```

421     RETARDO:
422         ldi r18, 20
423
424     RET1:
425         ldi r19, 255
426
427     RET2:
428         ldi r20, 255
429
430     RET3:
431         dec r20
432         brne RET3
433
434         dec r19
435         brne RET2
436
437         dec r18
438         brne RET1
439
440         ret

```

Fig. 10. Subrutina utilizada para generar el retardo entre patrones.

Como se observa en la Fig. 9, los pulsadores son leídos mediante el registro **PINB**. Debido al uso de resistencias pull-up, una pulsación produce un nivel lógico bajo. Para el primer botón se incrementa **r21**, permitiendo avanzar a la siguiente secuencia; si se supera el valor 8, el registro vuelve a 1. El segundo botón realiza la operación contraria mediante **DEC**, regresando a la secuencia 8 cuando el valor disminuye por debajo de 1. La espera de liberación del pulsador evita que una misma pulsación sea interpretada repetidamente.

El tercer pulsador carga directamente el valor 1 en **r21**, permitiendo regresar a la secuencia inicial.

Como se observa en la Fig. 10, la temporización entre los diferentes patrones se implementó mediante una subrutina de retardo por software. Para ello se utilizaron los registros **r18**, **r19** y **r20** como contadores de tres ciclos anidados. Cada contador es decrementado mediante **DEC** y la instrucción **BRNE** mantiene el ciclo mientras el resultado sea distinto de cero. Esta subrutina es llamada mediante **RCALL** desde las diferentes secuencias.

TABLE V
DESCRIPCIÓN DE LAS SECUENCIAS IMPLEMENTADAS

Secuencia	Patrón implementado
1	Desplazamiento de un LED hacia la izquierda.
2	Desplazamiento de un LED hacia la derecha.
3	Alternancia entre los patrones 10101010 y 01010101.
4	Movimiento de los LEDs desde los extremos hacia el centro.
5	Movimiento de los LEDs desde el centro hacia los extremos.
6	Encendido progresivo de los ocho LEDs.
7	Apagado progresivo de los ocho LEDs.
8	Encendido y apagado simultáneo de los ocho LEDs.

Las ocho secuencias implementadas se resumen en la Tabla V. Cada una utiliza diferentes patrones binarios para controlar el estado de los LEDs conectados al **PORTD**.

D. ALU

1) *Desarrollo del código en ensamblador:* Para implementar la Unidad Aritmético-Lógica se utilizó el microcontrolador ATmega328P programado en lenguaje ensamblador. El sistema recibe dos operandos de 4 bits, A y B, y un código de selección S de 3 bits. Según el valor de S se ejecuta una de las ocho operaciones solicitadas: CLEAR, resta, suma, XOR, AND, OR, desplazamiento a la izquierda e incremento. Para organizar el programa se utilizaron registros de propósito general. R16 almacena A, R17 almacena B, R18 contiene el selector S, R19 guarda el resultado F y R20 almacena las banderas con la organización 00000NZN. R21 y R22 se utilizaron como registros auxiliares para la lectura y el tratamiento de las entradas. El PORTD se utilizó para recibir los ocho bits de A y B, mientras que PC0, PC1 y PC2 del PORTC reciben el selector S. Las salidas del resultado y las banderas se manejan desde PORTB, y PC3, PC4 y PC5 se utilizan para mostrar el código de operación seleccionado.

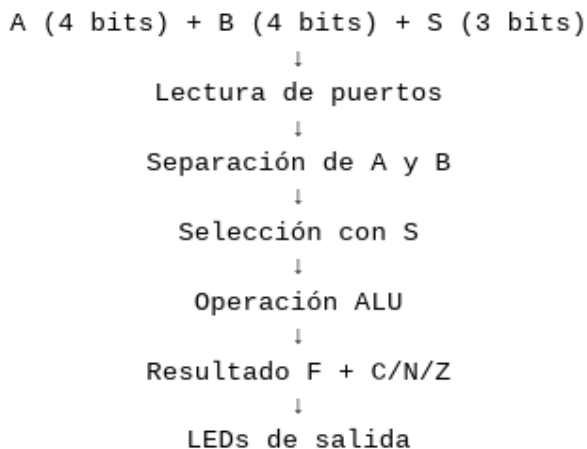


Fig. 11. diagrama de bloques general de la ALU: entradas A/B/S → lectura GPIO → selección → operación → banderas → LEDs.

2) *Lectura de entradas y seleccion de operacion:* Los operandos A y B se leen juntos desde PORTD. Los cuatro bits inferiores corresponden a A y los cuatro superiores a B. Para separar la información se utilizan máscaras con ANDI. En el caso de B también se utiliza SWAP, que intercambia los cuatro bits superiores con los inferiores para dejar el operando en una posición cómoda para realizar las operaciones. El selector S se lee desde los tres bits inferiores de PORTC. Su valor se compara con los números de 0 a 7 mediante CPI y, cuando existe coincidencia, BREQ dirige la ejecución hacia la rutina correspondiente. De esta forma se selecciona una de las ocho operaciones de la ALU.

```

; LEER A Y B DESDE PORTD
IN R21, PIND          ; Lee los 8 switches de PORTD
MOV R16, R21          ; Copia el puerto para obtener A
ANDI R16, 0b00001111 ; Conserva PD0-PD3 -> A

LDI R22, 0b00000001   ; Carga una máscara que apunta solamente al bit A0
EOR R16, R22          ; Invierte solo A0 porque físicamente ahora usa pull-up

MOV R17, R21          ; Copia nuevamente el puerto para obtener B
SWAP R17              ; Baja B desde bits 7-4 hacia bits 3-0
ANDI R17, 0b00001111 ; Conserva solamente B

; LEER S DESDE PORTC
IN R18, PINC          ; Lee PORTC
ANDI R18, 0b00000111 ; Conserva solamente PC0-PC2 -> S

```

Fig. 12. Fragmento de código donde se leen A, B y S, mostrando ANDI, SWAP y la lectura con IN.

```

; SELECCION DE LA OPERACION

CPI R18, 0          ; Compara S con 0
BREQ OP_CLEAR      ; Si si vale 0 salta a OP_CLEAR

CPI R18, 1          ; Compara S con 1
BREQ OP_SUB        ; Si si vale 1 salta a OP_SUB

CPI R18, 2          ; Compara S con 2
BREQ OP_ADD        ; Si si vale 2 salta a OP_ADD

CPI R18, 3          ; Compara S con 3
BREQ OP_XOR        ; Si si vale 3 salta a OP_XOR

CPI R18, 4          ; Compara S con 4
BREQ OP_AND        ; Si si vale 4 salta a OP_AND

CPI R18, 5          ; Compara S con 5
BREQ OP_OR         ; Si si vale 5 salta a OP_OR

CPI R18, 6          ; Compara S con 6
BREQ OP_SHL        ; Si si vale 6 salta a OP_SHL

CPI R18, 7          ; Compara S con 7
BREQ OP_INC        ; Si si vale 7 salta a OP_INC

```

Fig. 13. Fragmento corto con CPI/BREQ mostrando cómo se elige la operación según S.

3) *Resultados y banderas:* Cada operación guarda su resultado en R19. Como los registros del ATmega328P son de 8 bits y la ALU trabaja con resultados de 4 bits, antes de mostrar la salida se conserva únicamente el nibble inferior mediante una máscara 00001111. Las banderas se guardan

TABLE VI
OPERACIONES DE LA ALU SEGÚN LAS ENTRADAS DE SELECCIÓN
 $S_2S_1S_0$.

S2	S1	S0	Operación
000			CLEAR
001			A - B
010			A + B
011			XOR
100			AND
101			OR
110			SHL
111			INC

en R20 con la organización 00000NZC. Carry se utiliza para indicar un acarreo o préstamo según la operación, Zero se activa cuando el resultado de 4 bits es 0000 y Negative se obtiene a partir del bit 3 del resultado. En la implementación física F3 y N comparten la misma señal, ya que N se obtiene directamente de ese bit. Finalmente, el resultado F y las banderas se envían a los LEDs mediante PORTB. El código S también se presenta en tres salidas para poder identificar visualmente qué operación se encuentra seleccionada.

4) *Simulación en PICSimLab*: Una vez ensamblado el programa en Microchip Studio se generó el archivo .hex y se realizó una simulación previa en PICSimLab utilizando la placa Arduino Uno. Se agregaron once interruptores para representar A, B y S, y LEDs para visualizar el resultado, las banderas y el código de operación. Durante la simulación se probaron distintas combinaciones. Por ejemplo, para A=0101, B=0011 y S=010 se obtuvo F=1000, correspondiente a la suma. También se verificó el caso A=1111, B=0001 y S=010, donde se obtuvo F=0000 con Carry y Zero activados. Estas pruebas permitieron comprobar el programa antes de pasar al montaje físico.

5) *Implementación física*: Después de verificar la simulación se realizó el montaje físico con Arduino Uno, DIP-switches, LEDs y resistencias. Los operandos A y B utilizan ocho entradas digitales y el selector S utiliza tres entradas adicionales. La mayoría de los switches se conectaron con resistencias pull-down para que un switch apagado represente 0 y uno activado represente 1. Durante las primeras pruebas se detectó una lectura incorrecta en A0, conectado al pin D0/RX. Al medir la tensión se comprobó que ese pin no alcanzaba correctamente el nivel bajo esperado. Para resolverlo se utilizó una conexión pull-up solamente en A0 y se invirtió ese bit por software mediante EOR y el registro auxiliar R22. Después de esta corrección la lectura funcionó de forma estable. Los LEDs de salida se conectaron con resistencias limitadoras de corriente y permiten visualizar F, las banderas y el código de selección.

VII. RESULTADOS Y DISCUSIÓN

A. Lavadora Automática

Las pruebas realizadas permitieron comprobar el funcionamiento de la secuencia completa de la lavadora automática. Al iniciar el sistema se encienden los LEDs de listo y carga ligera. Antes de comenzar el proceso, el pulsador de

selección permite cambiar entre carga ligera, media y pesada, encendiéndose en cada caso el indicador correspondiente.

Luego de presionar START, el programa permanece esperando hasta detectar la puerta cerrada. Una vez cumplida esta condición, se espera la señal correspondiente al nivel de agua. El lavado no comienza hasta que ambas condiciones se hayan cumplido.

Al iniciar el lavado se apaga el LED de listo y se enciende el LED de lavado. Se verificaron las cinco repeticiones de giro y pausa programadas. Una vez completadas, el sistema cambia automáticamente al centrifugado y posteriormente al secado. Durante esta última etapa se comprobó el giro hacia la derecha, la pausa y el giro hacia la izquierda. Finalmente se apagan los indicadores de proceso y se enciende el LED de fin.

TABLE VII
DURACIÓN APROXIMADA DEL CICLO SEGÚN LA CARGA

Carga	Lavado	Centrif.	Secado	Total
Ligera	15 s	15 s	13 s	43 s
Media	25 s	18 s	19 s	62 s
Pesada	35 s	21 s	25 s	81 s

Los tiempos obtenidos se resumen en la Tabla VII. Para la carga ligera el proceso completo desde el comienzo del lavado dura aproximadamente 43 segundos, mientras que para las cargas media y pesada aumenta a aproximadamente 62 y 81 segundos, respectivamente.

Durante la simulación también se observó que el pulsador virtual de selección de carga podía presentar cambios rápidos entre opciones. Se revisó su configuración para trabajar como entrada activa en nivel bajo, de acuerdo con las resistencias pull-up utilizadas. El resto de los estados y transiciones presentó el comportamiento esperado durante las pruebas.

En general, las pruebas permitieron verificar correctamente la selección de carga, los estados del proceso, la lectura de los sensores, las temporizaciones, los LEDs indicadores y la representación del movimiento del motor. La simulación permitió comprobar previamente la secuencia y posteriormente se verificó el funcionamiento del sistema implementado.

B. Contador Digital

La validación del módulo de conteo digital demostró una correcta integración entre el software desarrollado en ensamblador y el hardware implementado. Mediante la decodificación por tabla de búsqueda proyectada sobre el Puerto D, se consiguió un encendido correcto de los segmentos del display para la representación visual de los dígitos del 0 al 9.

En cuanto a las señales de entrada, la lógica de control respondió según las condiciones de diseño programadas:

- **Gestión de pulsadores y límites:** Las rutinas asociadas a los pines del Puerto B procesaron de forma precisa las órdenes de incremento y decremento. Se verificó experimentalmente el bloqueo de la cuenta en los extremos, impidiendo el desbordamiento hacia valores superiores a 9 o inferiores a 0. Del mismo modo, la lectura del pin

de *reset* forzó de manera efectiva el reinicio del registro contador.

- **Estabilidad temporal y anti-rebote:** Las subrutinas de retardo por software eliminaron los ruidos mecánicos generados por los conmutadores, evitando la detección de múltiples pulsaciones espurias en un solo evento de activación.

El comportamiento del algoritmo se evaluó con éxito de forma preliminar mediante simulación en PicSimLab y posteriormente se corroboró sobre el microcontrolador ATmega328P físico. Los soportes gráficos de las pruebas y la secuencia de funcionamiento se registran detalladamente en el Apéndice del documento.

C. Selección y ejecución de secuencias

Como resultado de la implementación, se logró ejecutar correctamente el sistema de selección y control de las ocho secuencias de LEDs. Cada una de las secuencias presentó el comportamiento esperado, permitiendo visualizar de forma clara los diferentes patrones de encendido, apagado y desplazamiento implementados.

El funcionamiento de los tres pulsadores también fue satisfactorio, permitiendo avanzar hacia la siguiente secuencia, retroceder a la anterior y regresar directamente a la secuencia inicial. Asimismo, los retardos implementados permitieron observar de forma adecuada la transición entre los distintos patrones de LEDs.

Finalmente, las pruebas realizadas permitieron comprobar que el programa respondió correctamente ante las diferentes entradas y que cada secuencia cumplió con el funcionamiento establecido. Las evidencias correspondientes al funcionamiento del sistema, incluyendo imágenes y registros audiovisuales, se encuentran disponibles en el material complementario indicado en el Apéndice.

D. ALU

La ALU implementada permitió ejecutar correctamente las ocho operaciones definidas mediante el selector de 3 bits. Los operandos A y B fueron interpretados como valores de 4 bits y el resultado F se presentó mediante cuatro salidas digitales. Las pruebas permitieron verificar también las banderas Carry, Negative y Zero. En casos donde el resultado superó los 4 bits se conservaron los cuatro bits menos significativos y se activó Carry. Por ejemplo, para 15+1 se obtuvo F=0000, C=1 y Z=1. Para una resta con préstamo, como 1-2, se obtuvo F=1111, C=1, N=1 y Z=0. La simulación en PICSimLab y el montaje físico mostraron el mismo comportamiento general. La principal dificultad apareció en la entrada A0 por el uso del pin D0/RX; el cambio a pull-up y la inversión por software permitieron corregir el problema sin rehacer el resto del circuito.

El desarrollo de la ALU fue versionado mediante Git y almacenado en el repositorio público del laboratorio en GitHub. Los commits realizados permiten observar la evolución desde las primeras pruebas de las operaciones hasta la lectura de

TABLE VIII
TABLA DE PRUEBAS DE LAS OPERACIONES DE LA ALU.

A	B	S	Operación	F	C	N	Z
0101	0011	000	CLEAR	0000	0	0	1
0001	0010	001	A-B	1111	1	1	0
1111	0001	010	A+B	0000	1	0	1
0101	0011	011	XOR	0110	0	0	0
0101	0011	100	AND	0001	0	0	0
0101	0011	101	OR	0111	0	0	0
1001	—	110	SHL	0010	1	0	0
1111	—	111	INC	0000	1	0	1

entradas reales, la incorporación de salidas y la corrección realizada durante la implementación física.

TABLE IX
HISTORIAL DE CAMBIOS Y ETAPAS DE DESARROLLO.

Commit / etapa	Qué cambió
Commit real 1	Prueba inicial o primeras operaciones de la ALU.
Commit real 2	Implementación de las ocho operaciones y/o banderas.
Commit real 3	Lectura de A, B y S desde los puertos.
Commit real 4	Salidas físicas y visualización del selector.
Commit real 5	Corrección de A0 para la implementación física.

VIII. CONCLUSIONES

- La implementación de la lavadora automática permitió aplicar una máquina de estados para organizar un proceso secuencial con diferentes condiciones de entrada. La combinación de sensores, selección de carga, temporización y LEDs permitió controlar de forma ordenada las etapas de lavado, centrifugado y secado mediante lenguaje ensamblador.
- Se logró la correcta inicialización y control de los registros de dirección de los puertos (como DDR_x y PORT_x) del microcontrolador ATmega328P, lo que permitió establecer una interfaz estable y bidireccional con los periféricos externos (pulsadores, LEDs y display de cátodo común).
- La implementación de instrucciones aritmético-lógicas (ANDI, CPI), de desplazamiento, saltos condicionales (BREQ) y retardos por software facilitó el desarrollo de una lógica de control eficiente para la ALU y el contador, optimizando el procesamiento directo mediante registros sin depender de bibliotecas de alto nivel.
- La verificación en simulador y su posterior montaje físico confirmaron la correcta respuesta del sistema ante las diferentes combinaciones de entrada, validando tanto el manejo de las banderas de estado (C, N, Z) como la efectividad de los filtros de antirrebote para una lectura precisa de los pulsadores.

IX. APÉNDICE

Nota: Como material complementario al presente informe se incluyen los códigos desarrollados, simulaciones,

fotografías y registros audiovisuales correspondientes a las diferentes actividades realizadas durante el laboratorio.

El material complementario se encuentra organizado en los siguientes repositorios digitales:

- **Repositorio de GitHub:** contiene los códigos fuente desarrollados en lenguaje ensamblador para el ATmega328P, junto con el historial de modificaciones realizadas durante el desarrollo del laboratorio.
- **Repositorio de Google Drive:** contiene las evidencias complementarias de las actividades realizadas, incluyendo capturas de las simulaciones, fotografías de los montajes y registros audiovisuales del funcionamiento de los sistemas implementados.

El código fuente del proyecto puede consultarse en:

Repositorio de GitHub (clic aquí)

Las evidencias y el material audiovisual pueden consultarse en:

Repositorio de Google Drive (clic aquí)

REFERENCES

- [1] Microchip Technology Inc., "ATmega328P Datasheet," [En línea]. Disponible en: <https://www.alldatasheet.es/datasheet-pdf/view/1425005/MICROCHIP/ATMEGA328P.html>
- [2] itemis AG, "What is a State Machine?," [En línea]. Disponible en: https://www.itemis.com/en/products/itemis-create/documentation/user-guide/overview_what_are_state_machines
- [3] Pico Technology, "What is Switch Bounce & How to Implement Debounce," [En línea]. Disponible en: <https://www.picotech.com/library/articles/blog/what-is-switch-bounce-how-to-implement-debounce>
- [4] SunFounder, "Display de 7 Segmentos," [En línea]. Disponible en: https://docs.sunfounder.com/projects/3in1-kit-v2/es/latest/components/component_7_segment.html